

Name: Shreya Gholase
PRN NO: 23070521145

Practical 07

Write and execute PL/SQL function to print /return binary equivalent of decimal number.

Introduction

A PL/SQL function is a subprogram that computes and returns a value. It helps in reusability, modular programming, and efficient database operations.

Key Concepts Used in This Program

- Functions in PL/SQL: A function must have a return type and return a value.
- Loops in PL/SQL: We use loops to repeatedly divide the decimal number by 2 to obtain its binary equivalent.
- String Operations: We build the binary number as a string.

PL/SQL Function to Convert Decimal to Binary

Steps to Convert Decimal to Binary in PL/SQL

1. Take a decimal number as input.
2. Use a LOOP to repeatedly divide the number by 2.
3. Store the remainders (0 or 1) in reverse order.

4. Return the final binary string.

PL/SQL Function Code

```
CREATE OR REPLACE FUNCTION decimal_to_binary(n IN NUMBER) RETURN
VARCHAR2 IS
    binary_result VARCHAR2(100) := ''; -- Variable to store the
    binary equivalent
    num NUMBER := n; -- Copy of the input number
    remainder
NUMBER; -- Stores remainder after division
BEGIN
    -- Check for zero case
    IF num = 0 THEN
        RETURN '0';
    END IF;

    -- Loop to convert decimal to binary
    WHILE num > 0 LOOP
        remainder := MOD(num, 2); -- Get remainder when divided by 2
        binary_result := remainder || binary_result; -- Build binary
        string in reverse
        num := TRUNC(num / 2); -- Reduce number by dividing by 2
    END LOOP;

    RETURN binary_result; -- Return final binary value
END
decimal_to_binary;
/
```

How to Execute the Function

Call the Function Using PL/SQL Block

```
DECLARE

    decimal_num NUMBER := 10; -- Example decimal number
    binary_value VARCHAR2(100);

BEGIN

    binary_value := decimal_to_binary(decimal_num);
    DBMS_OUTPUT.PUT_LINE('Binary equivalent of ' || decimal_num || '
is: ' || binary_value);

END;

/
```

Expected Output:

Binary equivalent of 10 is: 1010

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2  decimal_num NUMBER := 10;
  3  binary_value VARCHAR2(100);
  4  BEGIN
  5  binary_value := decimal_to_binary(decimal_num);
  6  DBMS_OUTPUT.PUT_LINE('Binary equivalent of ' || decimal_num || ' is: ' || binary_value);
  7  END;
  8  /
Binary equivalent of 10 is: 1010
```

Explanation of the Code

Step	Description
------	-------------

Function Creation	Defines <code>decimal_to_binary</code> function with input <code>n</code> (decimal number).
Binary Result Variable	Stores the binary representation as a string.
Loop Execution	Repeatedly divides <code>num</code> by 2, storing remainders.

String Concatenation	Builds binary number in reverse order.
Return Statement	Returns the final binary string.

Task

1. Modify the function to display step-by-step conversion while calculating binary.

```

SQL> CREATE OR REPLACE FUNCTION decimal_to_binary(num IN NUMBER) RETURN VARCHAR2 IS
2     remainder NUMBER;
3     binary_result VARCHAR2(100) := '';
4     num_copy NUMBER;
5 BEGIN
6     num_copy := num;
7
8     IF num_copy = 0 THEN
9         DBMS_OUTPUT.PUT_LINE('Step 0: 0 / 2 = 0, Remainder = 0');
10        RETURN '0';
11    END IF;
12
13    DBMS_OUTPUT.PUT_LINE('Converting ' || num || ' to binary:');
14
15    WHILE num_copy > 0 LOOP
16        remainder := MOD(num_copy, 2);
17        binary_result := remainder || binary_result;
18        DBMS_OUTPUT.PUT_LINE('Step: ' || num_copy || ' / 2 = ' || TRUNC(num_copy / 2) ||
19                               ', Remainder = ' || remainder);
20        num_copy := TRUNC(num_copy / 2);
21    END LOOP;
22
23    RETURN binary_result;
24 END decimal_to_binary;
25 /

```

Function created.

```

SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
2     decimal_num NUMBER := 10;
3     binary_value VARCHAR2(100);
4 BEGIN
5     binary_value := decimal_to_binary(decimal_num);
6     DBMS_OUTPUT.PUT_LINE('Binary equivalent of ' || decimal_num || ' is: ' || binary_value);
7 END;
8 /

```

Converting 10 to binary:
Step: 10 / 2 = 5, Remainder = 0
Step: 5 / 2 = 2, Remainder = 1
Step: 2 / 2 = 1, Remainder = 0
Step: 1 / 2 = 0, Remainder = 1
Binary equivalent of 10 is: 1010

PL/SQL procedure successfully completed.

2. Write a PL/SQL block to accept user input for the decimal number and call the function.

```

SQL> SET SERVEROUTPUT ON;
SQL> ACCEPT decimal_num NUMBER PROMPT 'Enter a decimal number: '
Enter a decimal number: 23
SQL> DECLARE
  2  decimal_num NUMBER := &decimal_num;
  3  binary_value VARCHAR2(100);
  4  BEGIN
  5  binary_value := decimal_to_binary(decimal_num);
  6  DBMS_OUTPUT.PUT_LINE('Binary equivalent of ' || decimal_num || ' is: ' || binary_value);
  7  END;
  8  /
old  2:  decimal_num NUMBER := &decimal_num;
new  2:  decimal_num NUMBER :=          23;
Step-by-step conversion:
Step: 23 / 2 = 11, Remainder = 1
Step: 11 / 2 = 5, Remainder = 1
Step: 5 / 2 = 2, Remainder = 1
Step: 2 / 2 = 1, Remainder = 0
Step: 1 / 2 = 0, Remainder = 1
Binary equivalent of 23 is: 10111
PL/SQL procedure successfully completed.

```

3. Modify the function to store binary values in a table (**binary_conversions**).

```

SQL> SET SERVEROUTPUT ON;
SQL> ACCEPT decimal_num NUMBER PROMPT 'Enter a decimal number: '
Enter a decimal number: 32
SQL> DECLARE
  2     decimal_num NUMBER := &decimal_num;
  3     binary_value VARCHAR2(100);
  4 BEGIN
  5     binary_value := decimal_to_binary(decimal_num);
  6     DBMS_OUTPUT.PUT_LINE('Binary equivalent of ' || decimal_num || ' is: ' || binary_value);
  7 END;
  8 /
old  2:     decimal_num NUMBER := &decimal_num;
new  2:     decimal_num NUMBER :=          32;
Step-by-step conversion:
Step: 32 / 2 = 16, Remainder = 0
Step: 16 / 2 = 8, Remainder = 0
Step: 8 / 2 = 4, Remainder = 0
Step: 4 / 2 = 2, Remainder = 0
Step: 2 / 2 = 1, Remainder = 0
Step: 1 / 2 = 0, Remainder = 1
Binary equivalent of 32 is: 100000

PL/SQL procedure successfully completed.

SQL> SELECT * FROM binary_conversions;

DECIMAL_NUMBER
-----
BINARY_VALUE
-----
              0
0
              32
100000

```